



**Universidad
Tecnológica
del Perú**

Docente:

Carla María Santivañez Calderón

Curso:

Taller Programación (34777)

Tema:

Proyecto Final

Empresa:

“Descriptiva Consultor Inmobiliaria e IRL”

Integrantes:

Huaman Riojas, Leonardo Daniel **U23205177**

Núñez Cusihualpa, Jair Inti **U22321082**

Mendoza Manrique Gabriel Alexandro **U24221375**

Lima – 2025

Indice

- 1. INTRODUCCIÓN**
- 2. RESUMEN**
- 3. ASPECTOS GENERALES EMPRESARIALES**
 - 3.1. VISIÓN**
 - 3.2. MISIÓN**
 - 3.3. FODA**
 - 3.4. RESEÑA HISTÓRICA DEL NEGOCIO**
 - 3.5. ORGANIGRAMA**
 - 3.6. UBICACIÓN DE LA ORGANIZACIÓN (MAP GOOGLE)**
 - 3.7. FOTOS PANORÁMICAS DEL NEGOCIO Y ESTUDIANTES**
- 4. PROBLEMOLOGIA**
 - 4.1. PROBLEMA GENERAL**
 - 4.2. PROBLEMAS ESPECÍFICOS**
- 5. OBJETIVOS**
 - 5.1. OBJETIVO GENERAL**
 - 5.2. OBJETIVOS ESPECÍFICOS**
- 6. HIPÓTESIS**
 - 6.1. HIPÓTESIS GENERAL**
 - 6.2. HIPÓTESIS ESPECÍFICAS**
- 7. ASPECTOS INFORMÁTICOS Y RR. HH**
 - 7.1. ASPECTOS INFORMATICOS**
 - 7.1.1. HARDWARE**
 - 7.1.2. SOFTWARE**
 - 7.2. RECURSOS HUMANOS**
 - 7.2.1. RR. HH DE IMPLEMENTACIÓN**
 - 7.2.2. RR. HH NIVEL USUARIO**
 - 7.2.2.1. CLIENTE**
 - 7.2.2.2. SERVIDOR**
 - 7.3. FOTOS DE REALIDAD INFORMÁTICA**
- 8. MODELAMIENTO**
 - 8.1. ASPECTOS GENERALES DEL SISTEMA**
 - 8.1.1. DETALLE DEL SISTEMA**
 - 8.1.2. DETALLE DE OBJETOS**
 - 8.1.3. DETALLE DE MÉTODOS**
 - 8.1.4. DETALLE DE LOS, PROCESOS**
 - 8.1.5. DETALLE DE VARIABLES**
 - 8.2. DISEÑO**
 - 8.2.1. DIAGRAMA DE CASO DE USOS**
 - 8.2.2. DIAGRAMA DE SECUENCIA**
 - 8.2.3. DIAGRAMA DE COLABORACIÓN**
 - 8.2.4. DIAGRAMA DE FLUJO**
 - 8.2.5. DIAGRAMA DE CLASES**
 - 8.2.6. DIAGRAMA DE PAQUETES**
- 9. IMPLEMENTACIÓN DE LA APLICACIÓN**
 - 9.1. ALCANCES DE SOLUCIÓN**
 - 9.2. ESTRUCTURA GENERAL**
- 10. PROGRAMA PRINCIPAL**
 - 10.1. MENÚ**
 - 10.2. ESTRUCTURAS**
 - 10.3. CICLO IF-ELSE**
 - 10.4. CICLO WHILE-DO WHILE**
 - 10.5. JOptionPane**
- 11. EJECUCIÓN DEL PROGRAMA**
 - 11.1. INTERFASES**
 - 11.2. CÓDIGO**
- 12. CONCLUSIONES**
- 13. RECOMENDACIONES**
- 14. REFERENCIAS BIBLIOGRÁFICAS**
- 15. ANEXOS**

SISTEMA DE CONTROL DE MATERIALES

1. INTRODUCCIÓN

En el área de la construcción, una buena gestión de los materiales es fundamental para evitar perjuicios económicos y cumplir con los plazos de obra. Sin embargo, muchas empresas constructoras no disponen de un sistema automatizado para registrar: el ingreso de materiales, su uso por día y el desperdicio que se produce. Por esto, este proyecto está orientado a crear un sistema con el lenguaje Java para gestionar el inventario de los materiales y registrar su consumo según el proyecto y el cálculo del desperdicio con el fin de optimizar el uso de recursos, evitando gastos innecesarios y facilitando la toma de decisiones en línea.

2. RESUMEN.

El presente proyecto propone el desarrollo de un sistema en Java que permita el control de materiales en una constructora. El mismo permitirá registrar el ingreso de los materiales al almacén, llevar el control del mismo en cada uno de los proyectos y calcular el desperdicio generado y, también, generar reportes de stock para facilitar la reposición oportuna además de generar alertas en caso de que el stock almacenado baje de la cantidad mínima determinada.

La implantación de este sistema servirá para optimizar el control de los recursos, reducir las pérdidas económicas por desperdicio y mejorar la planificación de compras, contribuyendo a articular una gestión más eficiente en la obra de construcción.

3. ASPECTOS GENERALES EMPRESARIALES

3.1. Visión

Nuestra visión se centra en convertirnos en una empresa constructora de referencia en el mercado de Lima, orientándonos hacia la expansión a nivel nacional e internacional, proporcionando soluciones constructivas innovadoras y sostenibles. Nuestra aspiración se concreta en crecer consistentemente, a partir de la ejecución de proyectos de vivienda, reformaciones y reparaciones donde se incorporen el diseño, la funcionalidad, la tecnología de última generación.

Deseamos que nos conozcan como los que transforman espacios en entornos modernos y confortables, que satisfacen plenamente las necesidades de nuestros clientes y contribuyen al avance urbano y el progreso de la sociedad.

3.2. Misión

Nuestra misión consiste en favorecer el progreso de la sociedad mediante la ejecución de viviendas y proyectos de infraestructura que vayan más allá de las condiciones de los estándares tradicionales. Para nosotros es importante brindar posibilidades que incluyan la incorporación de técnicas avanzadas y sistemas de automatización, el acceso a los servicios modernos como internet, así como también espacios que incrementen la calidad de vida de nuestros clientes.

Más allá de construir nuevas viviendas nos especializamos en reparaciones y remodelaciones, adaptando edificaciones viejas a las demandas actuales y asegurando que cumplan con las exigencias que postula la globalización en todos sus aspectos. Trabajamos a nivel nacional, deseando reforzar nuestra presencia en Lima, aspirando incluso a un nivel internacional para la consecución de proyectos de calidad que propicien el bienestar, el confort y la mejora de las personas.

3.3. FODA

MATRIZ DE ANÁLISIS FODA (DAFO)

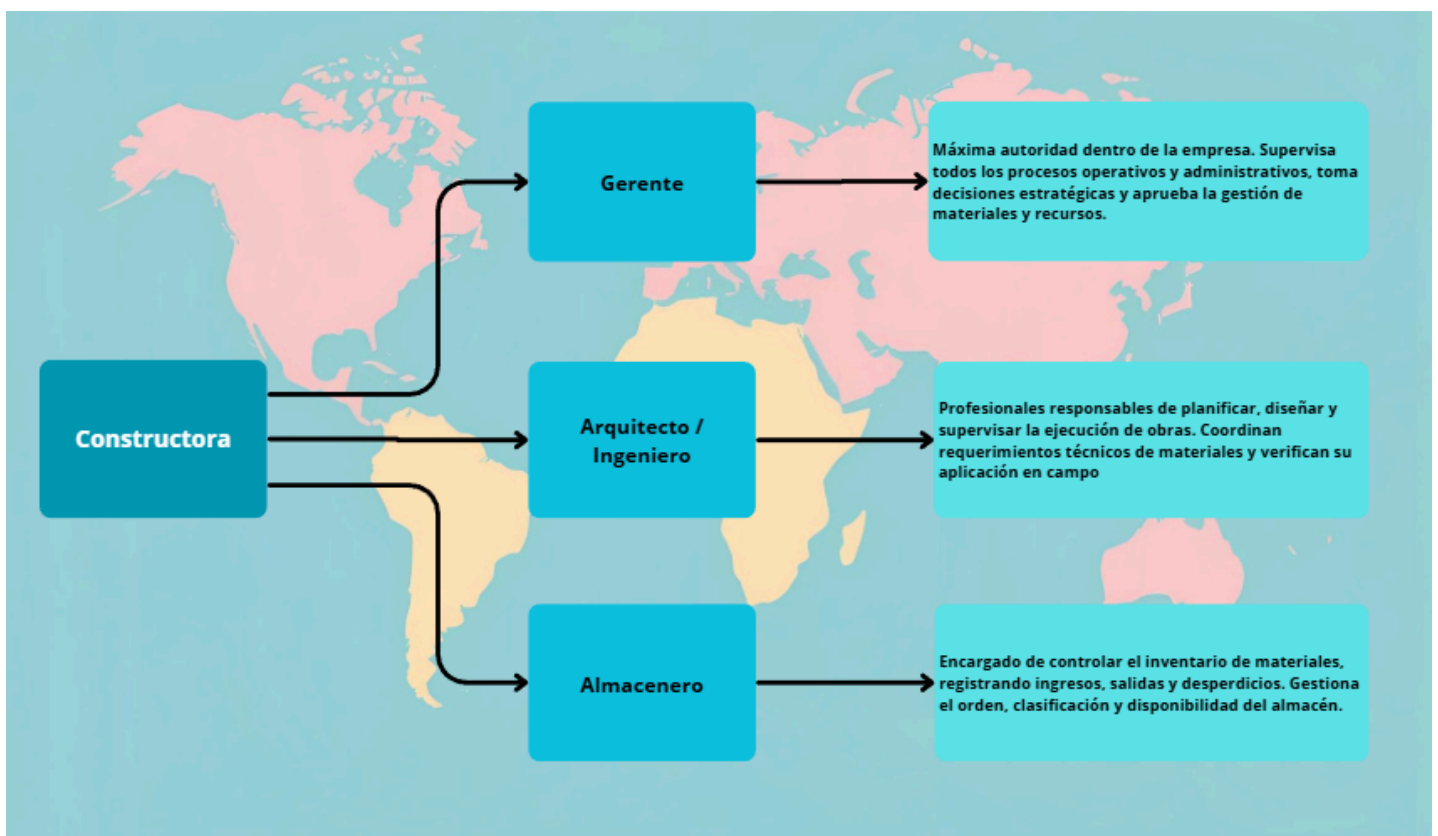


3.4. RESEÑA HISTÓRICA DEL NEGOCIO

La empresa Descriptiva Consultor Inmobiliaria E.I.R.L. se fundó en el 2023 por Cristian Gonzalo Núñez, en conjunto con el Grupo PIRKON. Desde su fundación, la empresa se ha orientado hacia la construcción, remodelación y desarrollo de proyectos inmobiliarios, realizando desde la parte de diseño y la elaboración de planos hasta su ejecución de obra y de entrega de unidades inmobiliarias con su correspondiente documentación legal.

La empresa tiene un plantel multidisciplinario de arquitectos e ingenieros que colaboran entre sí con el propósito de obtener proyectos de calidad. Hoy en día, uno de los principales retos de la empresa es mejorar su control de materiales, por lo cual está tratando de buscar un sistema digital que mejore su inventario y disminuya los desperdicios y aumente la eficiencia de los proyectos.

3.5. ORGANIGRAMA



3.6. UBICACIÓN DE LA ORGANIZACIÓN (MAP GOOGLE) (local de atención)



4. PROBLEMOLOGÍA

En el momento presente, la constructora presenta serias dificultades para controlar los materiales utilizados en el desarrollo de sus proyectos. Dicho proceso de gestión se lleva a cabo con la ayuda de plantillas de hojas de cálculo y registros en papel, y tal proceder causa problemas de inconsistencias y dificulta la trazabilidad de la información. Esto puede repercutir directamente en los costos de las obras, en la planificación y en el cumplimiento de los programas.

4.1. Problema General

Hoy en día la empresa constructora carece de un sistema automatizado para el registro, control y análisis de los materiales, lo que genera que los diversos tipos de información relacionada con los ingresos, el consumo y el desperdicio de materiales y/o insumos se manejen de forma manual, generando errores de registro, duplicidad de datos y pérdida de información clave.

La falta de control digital centralizado hace que sea más difícil saber qué materiales entran y salen del almacén, generando como consecuencia pérdidas económicas importantes, así como ampliar el desperdicio y retrasos en la ejecución de los proyectos. Además, la falta de información oportuna afecta a la planificación de las compras, generando sobrecostos y desorganización en la administración del almacén, afectando la rentabilidad de la empresa y su eficiencia operativa.

4.2. Problema específico

- **No existe un registro centralizado**, sino que la información de ingresos y egresos de materiales está dispersa en el sistema, lo cual dificulta tomar decisiones.
- **Falta de control del desperdicio**, el cual consiste en que no existe un seguimiento del material que se desperdicia para cada obra, lo cual impide identificar a qué área se le puede dar responsable para evitar desperdicios y así poder controlar costos.
- **Poca visibilidad del stock disponible** para los encargados de obra, donde no hay reportes en tiempo real, lo que provocará retrasos por falta de insumos o compras de emergencia.
- **Errores humanos de los reportes**, ya que se utilizan métodos manuales, donde a partir de los reportes se generan errores en las cantidades comunes, lo que deteriora la capacidad del inventario.
- **Sobrecostos de los proyectos**, ya que, debido a la falta de control, existe la posibilidad de que se compren materiales que no se usan y que se producen desperdicios y de uso de recursos.

5. OBJETIVOS

5.1. Objetivo General

Desarrollar e implementar un sistema informático de inventario en Java; que permita el registro, control y análisis del flujo integral de materiales de construcción: ingreso, uso y desperdicio; en tiempo real. Este sistema debe centralizar la información en una base de datos de confianza; permitiendo a los encargados de almacén, supervisores de obra, y gerencia, el acceso a los datos de forma ágil y confiable.

Con esta solución tecnológica se busca optimizar la administración de los recursos, reducir las pérdidas económicas futuras debido a las compras no necesarias y a las mermas demasiado grandes, y mejorar la trazabilidad de los materiales desde su adquisición hasta su utilización en obra. También, el sistema deberá permitir generar reportes y estadísticas que permitirán tomar decisiones en la alta dirección, lo que permitirá una mejor gestión; en suprema eficiencia, rentabilidad y en línea con el rumbo de la empresa constructora.

5.2. Objetivo Específico

Digitalizar el registro de materiales para eliminar el uso de hojas de cálculo y documentos físicos, minimizar el margen de error humano y concentrar toda la información en una base de datos confiable.

Implementar un módulo de control de stock que permita conocer en tiempo real los stocks de cada material, y que emita de forma automática las alertas cuando el stock se encuentre por debajo de un umbral mínimo predefinido.

Registrar y monitorear el consumo de materiales por obra mediante la obtención de informes por obra, por etapas de construcción y por responsable, facilitando la trazabilidad de los recursos.

Controlar y reducir el desperdicio de materiales mediante un registro adecuado de mermas y su análisis para realizar propuestas de mejora en la planificación y la ejecución de las obras.

Optimizar la toma de decisiones gerenciales mediante reportes gráficos y estadísticos, que permitan realizar la planificación de compras, el cálculo de costos y mejorar la rentabilidad de sus proyectos.

Conceder mayor seguridad y control sobre el acceso a la información mediante un sistema de usuarios mediante roles (administrador, almacenero, supervisor de obra), protegiendo la información y evitando modificaciones no autorizadas.

6. HIPÓTESIS

6.1. Hipótesis General

Si la constructora implementa un sistema de inventario desarrollado en Java, el cual será capaz de registrar, controlar y analizar en tiempo real lo que ingresa, lo que se usa y lo que se desperdicia, se espera que con esto se consiga una mejora muy significativa de la administración del almacén, que se reduzcan las pérdidas económicas y que se aumente la eficiencia de los proyectos de construcción.

Esto indica que se dispondrá de información centralizada y actualizada, que permite planificar las compras, que permite la identificación de los responsables en caso de desperdicio y la optimización del uso de los recursos que se utilizan en cada obra. Por lo tanto, ello deviene en una mayor rentabilidad de la constructora y una mejor obtención de los plazos en los proyectos.

6.2. Hipótesis Específicas

El registro de materiales automatizados disminuirá la capacidad de error humano y facilitará la mejora de la veracidad de los datos de inventario.

El control de stock en tiempo real potenciará decisiones rápidas, evitando retrasos por falta de insumos o compras de emergencia.

El análisis del desperdicio permitirá identificar fallas en los procesos de obra y facilitará la implementación de las acciones correctivas para disminuir costos.

La creación de los reportes estadísticos permitirá a la gerencia evaluar el desempeño de cada proyecto y mejorar la planificación futura.

La creación de los usuarios y roles dentro del sistema garantizará la seguridad de la información y la trazabilidad sobre los cambios aplicados.

7. ASPECTOS INFORMÁTICOS Y RRHH

7.1. ASPECTOS INFORMATICOS

7.1.1. Hardware

La empresa en la actualidad dispone de computadoras de escritorio, laptops y dispositivos móviles que permiten a los colaboradores del almacén y a los supervisores de la obra llevar el registro manual de los ingresos y egresos de materiales. Dichos equipos son necesarios para el funcionamiento del negocio, ya que son capaces de generar reportes de hojas de cálculo, enviar correos electrónicos y comunicaciones entre áreas.

Se reutilizarán los mismos equipos para la implementación del nuevo sistema siempre y cuando estos cumplan con al menos las especificaciones mínimas recomendadas: procesador i5 o en su defecto superior, 8 GB de memoria RAM, y acceso a internet para el trabajo en red.

7.1.2. Software

En la actualidad, las personas realizan el registro de materiales de forma manual, utilizando herramientas como Microsoft Excel y aplicaciones de mensajería como WhatsApp para coordinarse con los compañeros de trabajo en obra. Sin embargo, este procedimiento manual es propenso a errores y no permite centralizar la información de manera adecuada.

Por ello, se propone un sistema de información desarrollado en Java, con una interfaz gráfica de usuario amigable que permita gestionar de forma organizada el ingreso, uso y desperdicio de materiales. El sistema funcionará de manera local, utilizando únicamente funcionalidades propias de Java para el

almacenamiento y procesamiento de datos. Las fases de instalación y configuración serán realizadas por el equipo de desarrollo, que también capacitará a los usuarios sobre su funcionamiento. La implementación del sistema facilitará la generación automática de reportes y mejorará la eficiencia en la administración de los recursos.

7.2. Recursos Humanos

7.2.1. RR.HH. DE IMPLEMENTACIÓN

La instalación y la configuración del sistema serán tarea del equipo de desarrollo, en concreto de un miembro del grupo que se encargará de implementar la base de datos, compilar el código en Java y dejar el sistema operativo en los equipos de la empresa. Esta persona también estará encargada de efectuar pruebas iniciales, comprobar que funcione correctamente todas las funciones (registro de ingreso de material, control de consumo, reportes de stock) y formar al personal para que sea capaz de utilizar el sistema de forma productiva en su trabajo diario.

7.2.2. RR.HH. NIVEL USUARIO

- **Administrador de Obra:** Representa al usuario más avanzado que tendrá acceso para registrar el ingreso de materiales al almacén, aprobar su uso en obra, así como supervisar el stock que queda en cada proyecto.
- **Almacenero:** A su vez, llevará la contabilidad de los movimientos de ingreso, consumo y desperdicio de materiales y generará reportes de ese movimiento, en diarios o semanales, para informar a la gerencia.
- **Supervisor de Obra:** Consultará el sistema para conocer la disponibilidad de materiales a fin de planificar la actividad constructiva, solicitando en el sistema la reposición de los materiales cuando sea necesario.
- **Gerencia:** Consultará los reportes globales para analizar el consumo de materiales, evaluando el nivel de desperdicio y la posible toma de decisiones estratégicas en relación a las compras y a la planificación de los proyectos.
- La plataforma del sistema estará habilitada en máquinas portátiles y de escritorio conectadas a la red interna de la empresa. A la larga, podría pensarse en habilitar acceso también remoto para los supervisores de obra utilizando dispositivos móviles, permitiéndoles registrar los consumos directamente en campo.

8. MODELAMIENTO

8.1. Aspectos generales del sistema

8.1.1. Detalle del Sistema

El sistema implementa los siguientes módulos principales:

- **Registro de materiales:** Permite ingresar nuevos materiales al inventario.
- **Actualización de stock:** Modifica la cantidad disponible (ingreso, salida, desperdicio).
- **Listado de materiales:** Muestra todos los materiales registrados y sus características.
- **Identificación de Usuarios:** Solicita el nombre, apellido y DNI del usuario que usa la aplicación para colocarlo en el registro
- **Interfaz mediante JOptionPane:** Facilita la interacción mediante ventanas emergentes.
- **Menú:** Utiliza los arrays para poder mostrar las categorías existentes
- **Historial:** Muestra las ultimas actualizaciones realizadas

8.1.2. Detalle de Objetos:

A partir del código y el diseño encontrado en el documento, los principales objetos del sistema son:

- **Material:** Representa cada elemento almacenado, contiene información de stock, categoria, desperdicio y ultima actualización
- **Responsable:** Registra la identidad de la persona que está usando la aplicación, permite saber quien realiza cada acción con el sistema
- **Historial:** Guarda en memoria todas las acciones realizadas durante la sesión, registra fecha, responsable y la acción realizada
- **RegistrarMaterial:** Se encarga de agregar nuevos materiales al inventario, manda información al registro
- **ActualizarStock:** Añade, resta o registra el desperdicio de un material, actualiza la fecha de modificación y registra la acción en el historial
- **ListarMateriales:** Muestra todo el inventario y lo filtra por categoria

- **Menu:** Administra el flujo del sistema, también solicita los datos del responsable, controla el menú principal y gestiona las instancias del resto de clases

8.1.3. Detalle de los Métodos

A partir del análisis del código proporcionado, los métodos más importantes son:

- **Menu:** `main(String[] args)`
 - Crea las instancias de las clases, pide los datos del `Responsable`, y ejecuta el **ciclo `do-while`** con el **`switch`** para controlar el menú principal de opciones
- **Material:** Getter and Setters
 - Accede y modifica los atributos (`nombre`, `cantidad`, `categoria`, `stockMinimo`, `desperdicio`, `ultimaActualizacion`, `unidadMedida`)
- **Responsable:** `getNombreCompleto()`
 - Concatena el nombre y apellido para el registro de acciones.
- **Historial:** `registrarAccion(String texto)`, `mostrarHistorial()`
 - Almacena un registro de la acción realizada por un responsable en el arreglo `acciones`, también recorre el arreglo de acciones y lo muestra en una ventana de `JOptionPane`.
- **RegistrarMaterial:** `registrar(Material[] inv, int[] cont, Resp, Hist)`
 - Solicita datos del nuevo material, lo agrega al arreglo `inventario` y registra la acción en el `Historial`.
- **ActualizarStock:** `actualizar(Material[] inv, int cont, Resp, Hist)`
 - Busca el material por nombre y permite al usuario añadir, restar o registrar desperdicio. Actualiza la cantidad y lanza una ALERTA si el stock es menor al mínimo
- **ListarMateriales:** `listar(Material[] inv, int cont)`
 - Permite al usuario seleccionar un filtro (Ver todo, Materiales de Obra, Consumibles, Herramientas) y muestra los detalles en una lista
- **BuscarMaterial:** `buscar(Material[] inv, int cont)`
 - Solicita un nombre de material, lo busca en el arreglo y muestra sus detalles si es encontrado

8.1.4. Detalles de los procesos

Los procesos funcionales del sistema identificados son los siguientes:

- **Gestión de usuarios**
 - Solicitud de datos
 - Nombre
 - Apellido
 - DNI
- **Gestión del inventario**
 - Registro de nuevos materiales
 - Nombre del material
 - Designación de la U.Medida
 - Cantidad
 - Categoría
 - Stock Mínimo
 - Modificación del stock
 - Solicitud del Nombre del material
 - Añadir, Restar o determinar el desperdicio del material
- **Gestión de reportes internos**
 - Listado general de materiales
 - Consulta rápida del inventario
- **Gestión de navegación**
 - Control del menú principal
 - Control del menú de acceso

Todos estos procesos están implementados mediante:

- Diagramas de Clases
- Diagramas de Paquetes
- Clases Java
- Ciclos if-else
- Ciclos do-while
- JOptionPane

8.1.5. Detalle variables

A partir del código compartido, las variables más importantes y representativas son:

Variables del sistema en Menu

- Material[] inventario → Arreglo que almacena hasta 100 materiales.
- int[] contador → Controla cuántos materiales hay registrados.
- int opcion → Variable para manejar las opciones del menú.

Variables típicas en Material

- String nombre → Nombre del material.
- int cantidad → Cantidad en stock.
- String categoria → Tipo de material (De Obra, consumible o Herramienta).
- String unidadMedida .→ Unidad de medida del material

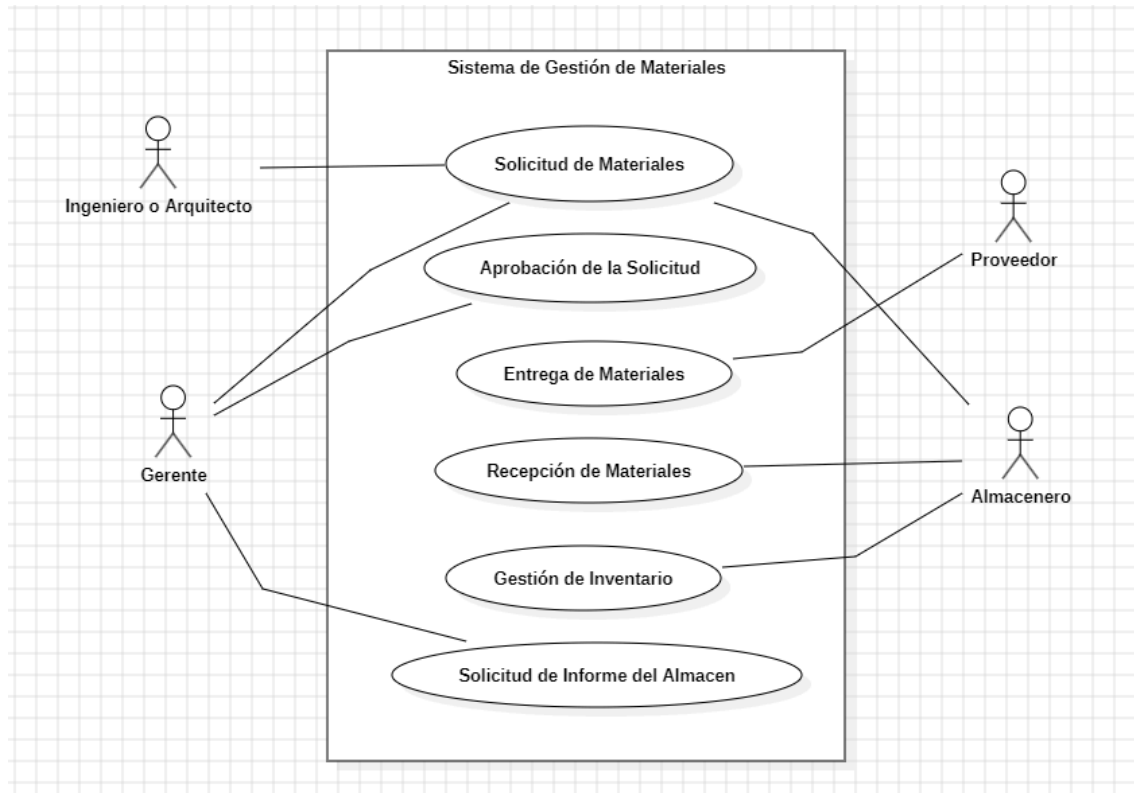
Variables en Responsable

- String nombre
- String apellido
- String dni

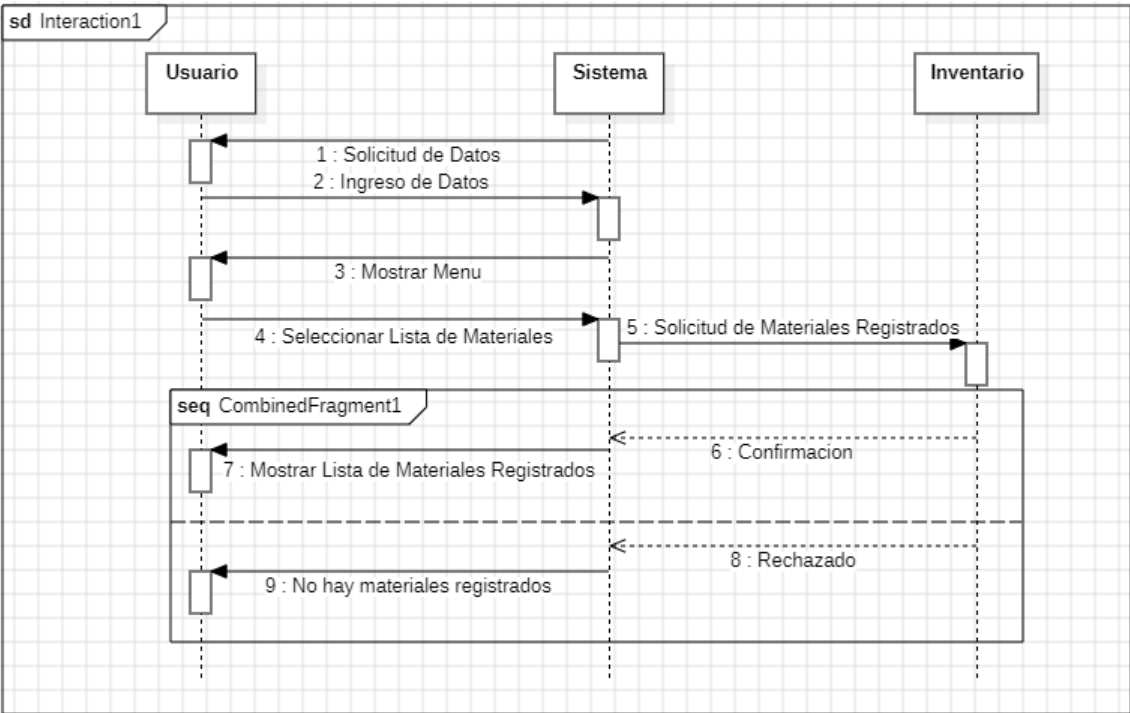
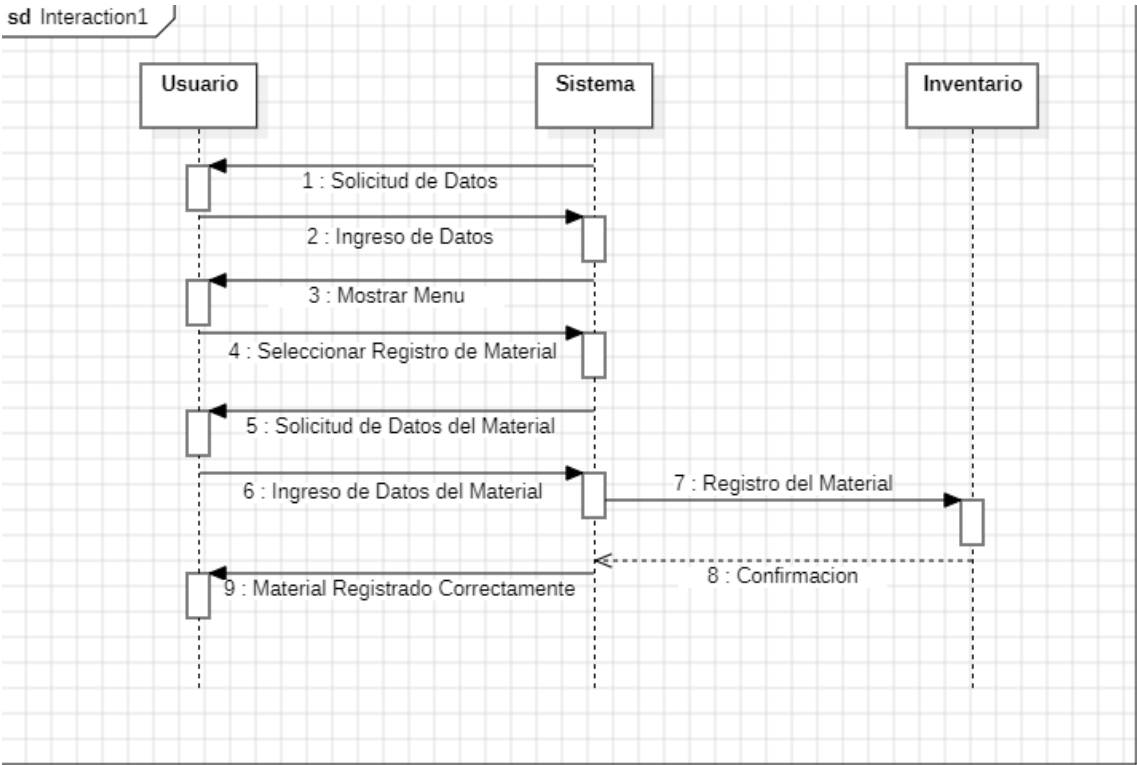
Estas variables permiten almacenar datos clave para ejecutar la lógica del sistema sin necesidad de una base de datos externa.

8.2. Diseño

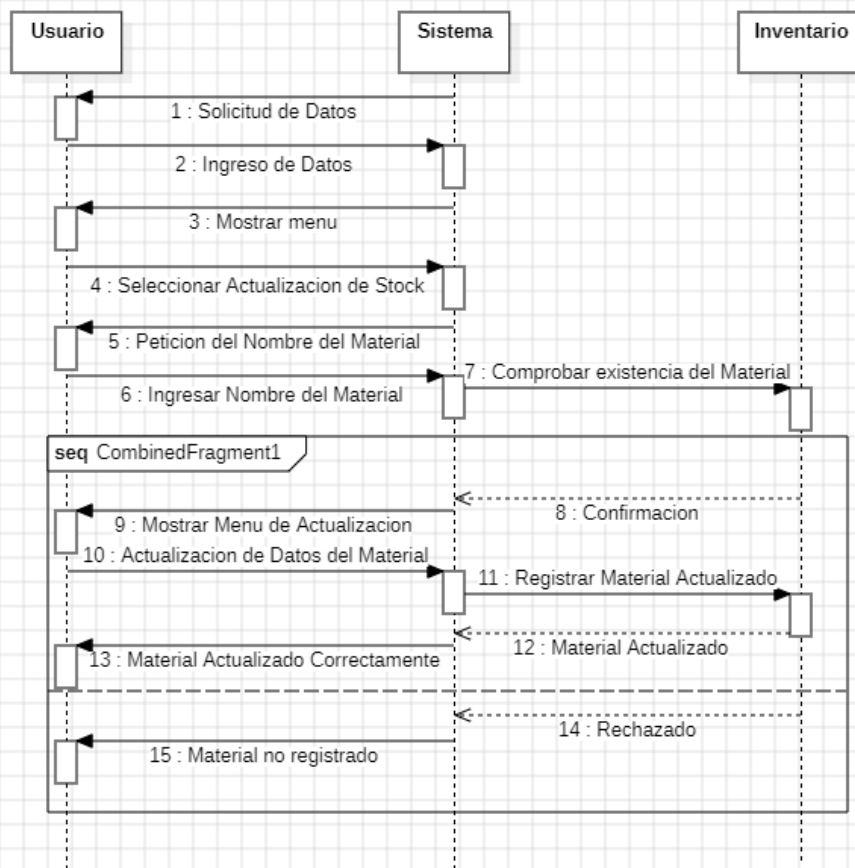
8.2.1. Diagrama de Casos de Uso



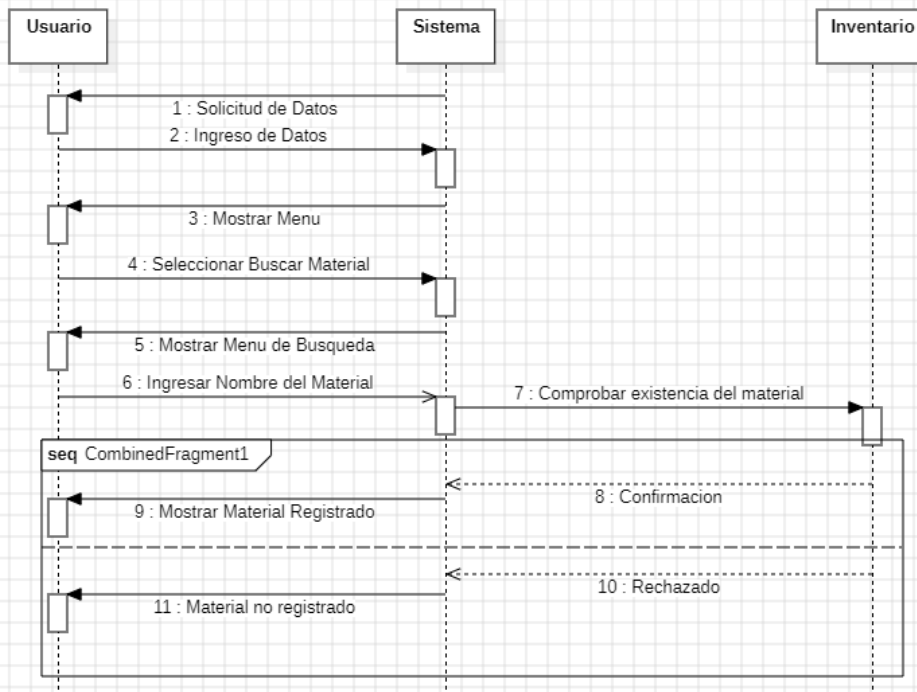
8.2.2. Diagrama de Secuencia

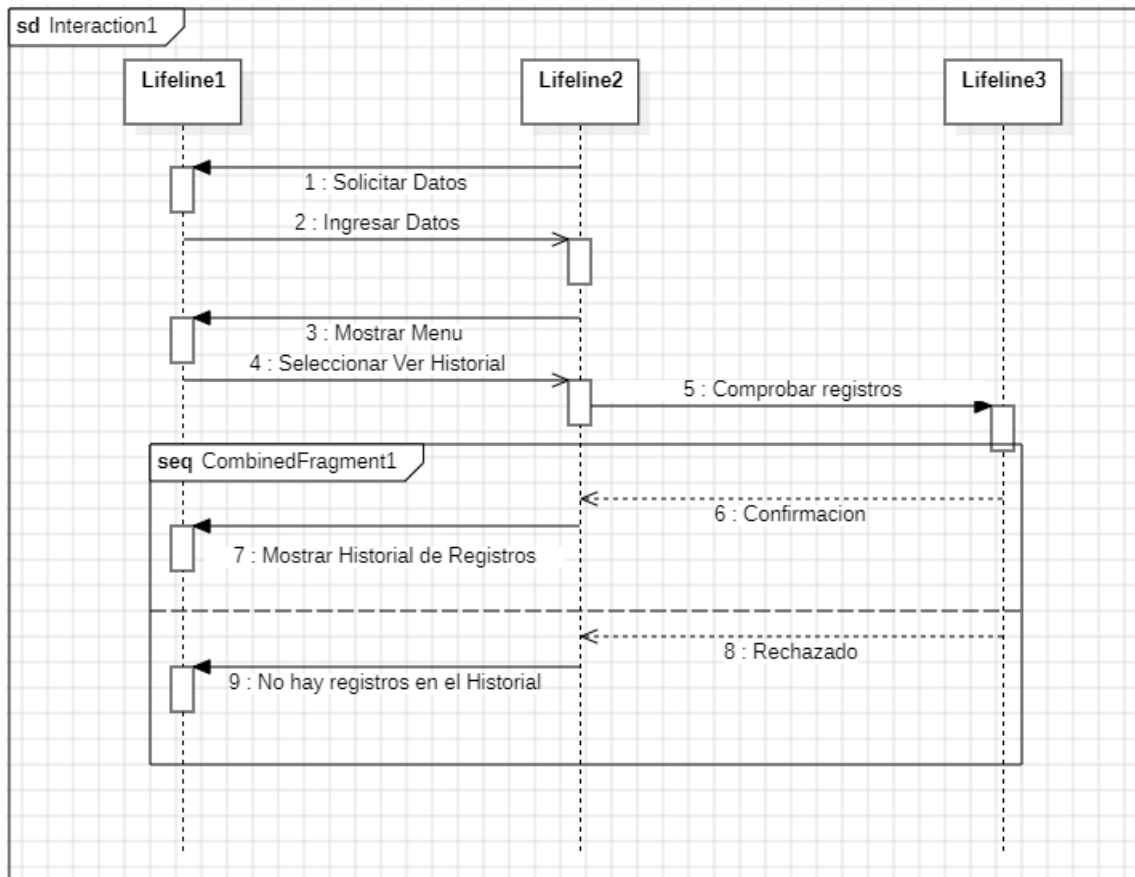


sd Interaction1

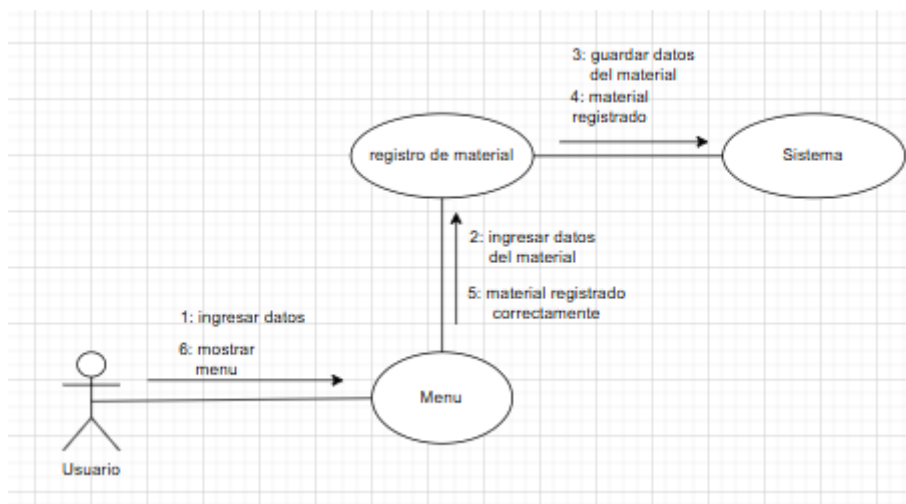


sd Interaction1

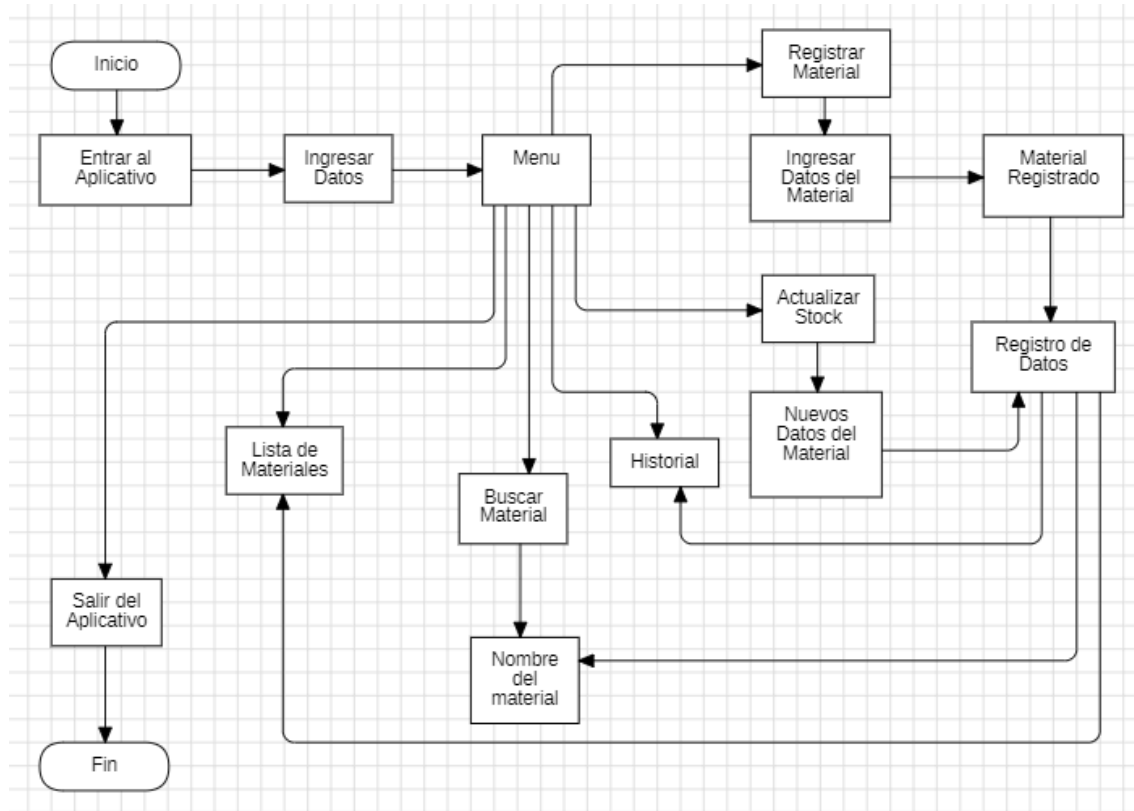




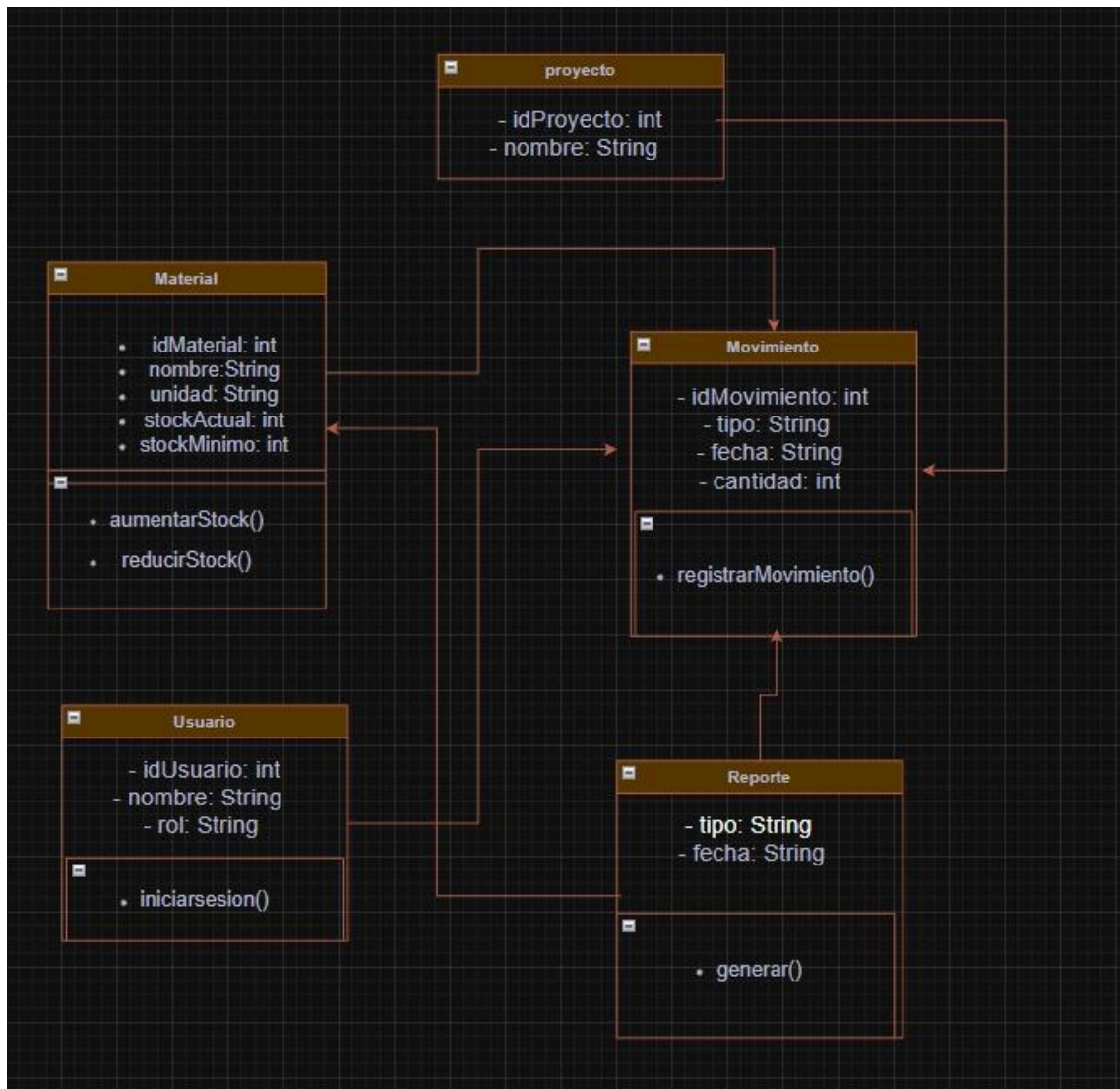
8.2.3. Diagrama de Colaboración



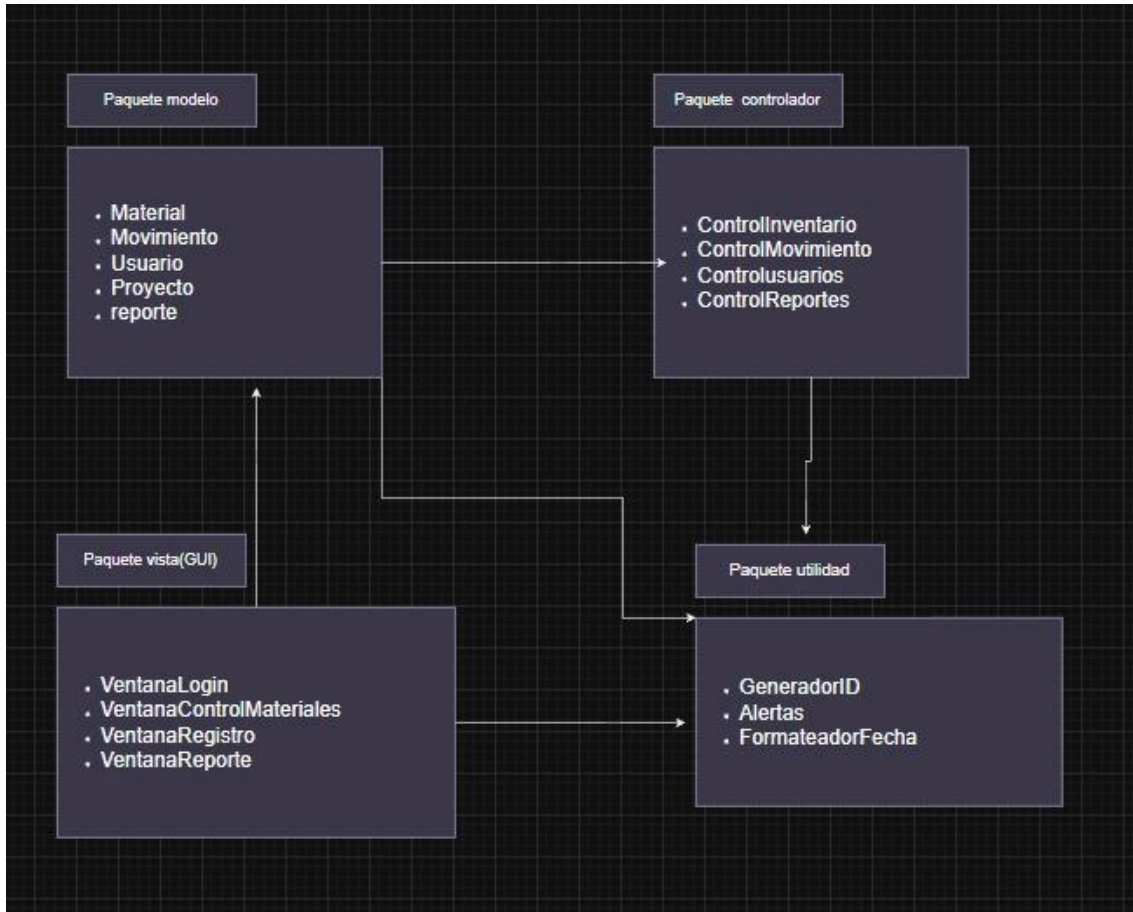
8.2.4. Diagrama de Flujo



8.2.5. Diagrama de Clases



8.2.6. Diagrama de Paquetes



9. Implementación de la aplicación

9.1. Alcances de solución

La herramienta creada facilita anotar la entrada de insumos, modificar las existencias actuales, supervisar la cantidad de existencias, documentar lo que se desecha y mostrar todos los elementos guardados en el programa. Además, maneja a las personas que lo usan mediante un método de alta y entrada para garantizar que solo los autorizados puedan acceder.

Esta aplicación se encarga de procesos que antes se hacían a mano (usando hojas de cálculo, papel o mensajes), lo que disminuye las equivocaciones y optimiza el orden de las existencias.

El programa carece de capacidades como la administración de dinero, conexión con otros programas, uso por internet, gestión automática de pedidos ni guardar información en grandes bases de datos, puesto que se construyó solo con Java usando elementos internos como listas.

9.2. Estructura General

El sistema está estructurado en los siguientes componentes:

- **Interfaz gráfica (GUI) mediante JOptionPane**

Se utiliza la librería estándar de Java para mostrar ventanas emergentes que permiten:

- Registrar materiales
- Actualizar stock
- Listar inventario
- Registrarse e iniciar sesión
- **Lógica de negocio organizada en clases**

El sistema está dividido en módulos representados por clases como:

- Main
- Usuario
- SistemaUsuarios
- Material
- RegistrarMaterial
- ActualizarStock
- ListarMateriales

Cada clase cumple una función específica dentro del flujo del sistema.

- **Almacenamiento interno con estructuras de Java (sin base de datos)**

Los materiales y usuarios se almacenan utilizando:

- Arreglos de objetos (`Material[] inventario`)
- Contadores para controlar la cantidad de registros

Esto permite mantener un sistema ligero y funcional sin depender de bases de datos externas.

- **Menús interactivos y controlados por ciclos**

El sistema utiliza menús implementados con:

- `if-else`
- `switch`
- `do-while`

Estos permiten navegar entre:

- Registro de usuarios
- Inicio de sesión
- Menú principal (registrar, actualizar, listar, salir)

10. Programa Principal

El programa principal controla el flujo del sistema mediante menús y manejo de sesiones. Se ejecutan los módulos de registrar materiales, actualizar stock y listar inventario según la opción seleccionada.

10.1. Menu

- Menú de inicio (registrarse, iniciar sesión)
- Menú principal (registrar, actualizar, listar, salir)

10.2. Estructura

Para la estructura se usa:

- Arreglos para almacenar materiales y usuarios
- Contadores para registro
- JOptionPane para la interfaz

10.3. Ciclo If-Else

Controla la validación de opciones ingresadas y manejo de errores simples.

10.4. Ciclo Do-While

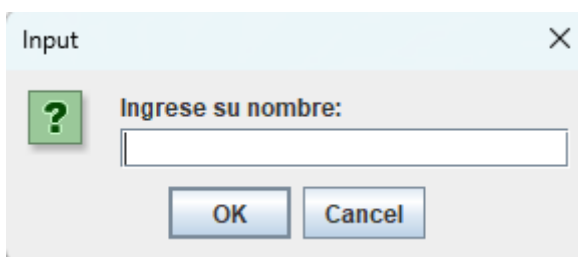
Controla la persistencia del menú hasta que el usuario seleccione la opción de salida.

10.5. JOptionPane

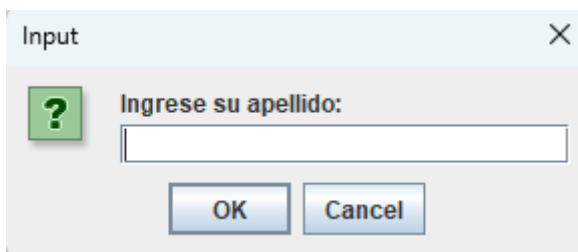
Permite la interacción con el usuario mediante cuadros de diálogo.

11. Ejecución del Programa

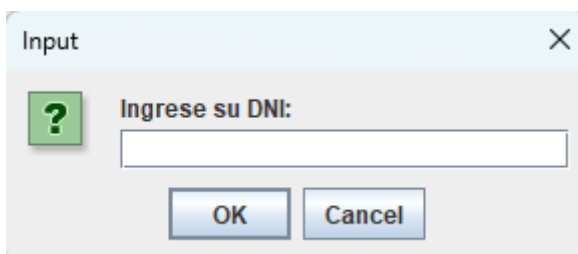
11.1. Interfases



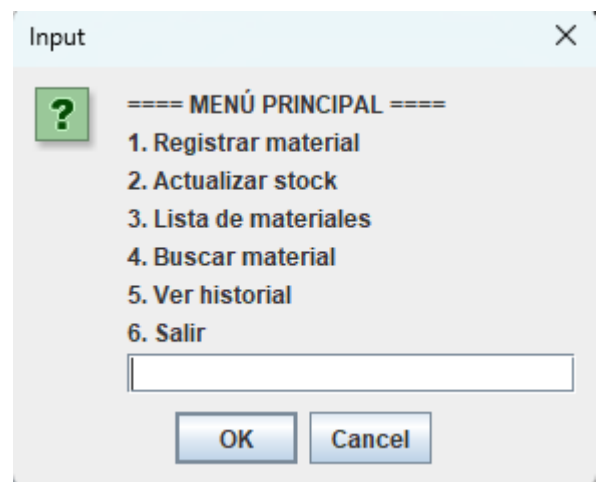
Input dialog box with a green question mark icon. The text "Ingrese su nombre:" is displayed above a text input field. Below the field are "OK" and "Cancel" buttons.



Input dialog box with a green question mark icon. The text "Ingrese su apellido:" is displayed above a text input field. Below the field are "OK" and "Cancel" buttons.



Input dialog box with a green question mark icon. The text "Ingrese su DNI:" is displayed above a text input field. Below the field are "OK" and "Cancel" buttons.



Input dialog box with a green question mark icon. The text "==== MENÚ PRINCIPAL =====" is displayed above a list of options: "1. Registrar material", "2. Actualizar stock", "3. Lista de materiales", "4. Buscar material", "5. Ver historial", and "6. Salir". Below the list is a text input field and "OK" and "Cancel" buttons.

11.2. Código

Clase Main: Menu

```
package com.mycompany.sistemagestiondealmacen;
import javax.swing.JOptionPane;

public class Menu {

    public static void main(String[] args) {

        Material[] inventario = new Material[100];
        int[] contador = {0};

        String nombre = JOptionPane.showInputDialog("Ingrese su nombre:");
        String apellido = JOptionPane.showInputDialog("Ingrese su apellido:");
        String dni = JOptionPane.showInputDialog("Ingrese su DNI:");
        Responsable responsable = new Responsable(nombre, apellido, dni);

        RegistrarMaterial reg = new RegistrarMaterial();
        ActualizarStock act = new ActualizarStock();
        ListarMateriales list = new ListarMateriales();
        BuscarMaterial buscar = new BuscarMaterial();
```

```

Historial historial = new Historial();

int opcion;

do {
    opcion = Integer.parseInt(JOptionPane.showInputDialog("""
        ==== MENÚ PRINCIPAL ====
        1. Registrar material
        2. Actualizar stock
        3. Lista de materiales
        4. Buscar material
        5. Ver historial
        6. Salir
        """));

    switch (opcion) {
        case 1 -> reg.registrar(inventario, contador, responsable, historial);
        case 2 -> act.actualizar(inventario, contador[0], responsable,
historial);
        case 3 -> list.listar(inventario, contador[0]);
        case 4 -> buscar.buscar(inventario, contador[0]);
        case 5 -> historial.mostrarHistorial();
        case 6 -> JOptionPane.showMessageDialog(null, "Fin del
programa.");
        default -> JOptionPane.showMessageDialog(null, "Opción inválida");
    }

    } while (opcion != 6);
}
}

```

Clase Material

```
package com.mycompany.sistemagestiondealmacen;
```

```
public class Material {
```

```
    private String nombre;
```

```
    private int cantidad;
```

```
    private String categoria;
```

```
    private int stockMinimo;
```

```
    private int desperdicio;
```

```
    private String ultimaActualizacion;
```

```
    private String unidadMedida;
```

```
    public Material(String nombre, int cantidad, String categoria, int stockMinimo,  
String unidadMedida) {
```

```
        this.nombre = nombre;
```

```
        this.cantidad = cantidad;
```

```
        this.categoria = categoria;
```

```
        this.stockMinimo = stockMinimo;
```

```
        this.unidadMedida = unidadMedida;
```

```
    }
```

```
    public Material(int desperdicio, String ultimaActualizacion) {
```

```
        this.desperdicio = desperdicio;
```

```
        this.ultimaActualizacion = ultimaActualizacion;
```

```
    }
```

```
    public String getNombre() {
```

```
        return nombre;
```

```
}
```

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}
```

```
public int getCantidad() {  
    return cantidad;  
}
```

```
public void setCantidad(int cantidad) {  
    this.cantidad = cantidad;  
}
```

```
public String getCategoria() {  
    return categoria;  
}
```

```
public void setCategoria(String categoria) {  
    this.categoria = categoria;  
}
```

```
public int getStockMinimo() {  
    return stockMinimo;  
}
```

```
public void setStockMinimo(int stockMinimo) {  
    this.stockMinimo = stockMinimo;  
}
```

```
public int getDesperdicio() {
```

```

        return desperdicio;
    }

    public void setDesperdicio(int desperdicio) {
        this.desperdicio = desperdicio;
    }

    public String getUltimaActualizacion() {
        return ultimaActualizacion;
    }

    public void setUltimaActualizacion(String ultimaActualizacion) {
        this.ultimaActualizacion = ultimaActualizacion;
    }

    public String getUnidadMedida() {
        return unidadMedida;
    }

    public void setUnidadMedida(String unidadMedida) {
        this.unidadMedida = unidadMedida;w
    }
}

```

Clase RegistrarMaterial

```

package com.mycompany.sistemagestiondealmacen;
import javax.swing.JOptionPane;
import java.time.LocalDate;

```

```

public class RegistrarMaterial {

    public void registrar(Material[] inventario, int[] contador, Responsable resp,
Historial historial) {

        if (contador[0] >= inventario.length) {
            JOptionPane.showMessageDialog(null, "Inventario lleno");
            return;
        }

        String nombre = JOptionPane.showInputDialog("Nombre del material:");

        String[] categorias = {"Materiales de Obra", "Materiales Consumibles",
"Herramientas"};

        String categoria = (String) JOptionPane.showInputDialog(
            null,
            "Seleccione la categoría:",
            "Categoría",
            JOptionPane.QUESTION_MESSAGE,
            null,
            categorias,
            categorias[0]
        );

        String unidad = JOptionPane.showInputDialog("Unidad de medida (kg,
unidades, litros, m³, etc.):");

        int cantidad = Integer.parseInt(JOptionPane.showInputDialog("Cantidad
inicial:"));

        int stockMinimo = Integer.parseInt(JOptionPane.showInputDialog("Stock
mínimo:"));

        inventario[contador[0]] =
            new Material(nombre, cantidad, categoria, stockMinimo, unidad);

```

```

        inventario[contador[0]].setUltimaActualizacion(LocalDate.now().toString());

        historial.registrarAccion(
            "[" + LocalDate.now() + "] " + resp.getNombreCompleto() +
            " (DNI " + resp.getDni() + ") registró " + nombre +
            " con cantidad " + cantidad + " " + unidad
        );

        contador[0]++;

        JOptionPane.showMessageDialog(null, "Material registrado
correctamente.");
    }
}

```

Clase ActualizarStock

```

package com.mycompany.sistemagestiondealmacen;

import javax.swing.JOptionPane;
import java.time.LocalDate;

public class ActualizarStock {

    public void actualizar(Material[] inventario, int contador, Responsable resp,
        Historial historial) {

        String buscar = JOptionPane.showInputDialog("Ingrese nombre del
material:");

        for (int i = 0; i < contador; i++) {

```

```

if (inventario[i].getNombre().equalsIgnoreCase(buscar)) {

    String opcion = JOptionPane.showInputDialog("""
        1. Añadir cantidad
        2. Restar cantidad
        3. Registrar desperdicio
        """);

    switch (opcion) {

        case "1" -> {

            int add =
Integer.parseInt(JOptionPane.showInputDialog("Cantidad a añadir:"));

            inventario[i].setCantidad(inventario[i].getCantidad() + add);
            inventario[i].setUltimaActualizacion(LocalDate.now().toString());

            historial.registrarAccion(
                "[" + LocalDate.now() + "]" +
                resp.getNombreCompleto() + " (DNI " + resp.getDni() + ")
añadió " +
                add + " unidades a " + inventario[i].getNombre() + ""
            );
        }

        case "2" -> {

            int restar =
Integer.parseInt(JOptionPane.showInputDialog("Cantidad a restar:"));

            if (restar > inventario[i].getCantidad()) {

                JOptionPane.showMessageDialog(null, "No puede restar más
de lo disponible.");

                return;
            }
        }
    }
}

```

```

    }

    inventario[i].setCantidad(inventario[i].getCantidad() - restar);
    inventario[i].setUltimaActualizacion(LocalDate.now().toString());

    historial.registrarAccion(
        "[" + LocalDate.now() + "]" +
        resp.getNombreCompleto() + " (DNI " + resp.getDni() + ")
restó " +
        restar + " unidades a " + inventario[i].getNombre() + ""
    );
}

case "3" -> {
    int desp =
Integer.parseInt(JOptionPane.showInputDialog("Cantidad desperdiciada:"));

    if (desp > inventario[i].getCantidad()) {
        JOptionPane.showMessageDialog(null, "Desperdicio mayor al
stock.");
        return;
    }

    inventario[i].setCantidad(inventario[i].getCantidad() - desp);
    inventario[i].setDesperdicio(inventario[i].getDesperdicio() +
desp);
    inventario[i].setUltimaActualizacion(LocalDate.now().toString());

    historial.registrarAccion(
        "[" + LocalDate.now() + "]" +
        resp.getNombreCompleto() + " (DNI " + resp.getDni() + ")
registró desperdicio de " +
        desp + " unidades de " + inventario[i].getNombre() + ""
    );
}

```

```

        );
    }

    default -> JOptionPane.showMessageDialog(null, "Opción
inválida");
}

    if (inventario[i].getCantidad() < inventario[i].getStockMinimo()) {
        JOptionPane.showMessageDialog(null,
            "ALERTA: El material " + inventario[i].getNombre() +
            " está por debajo del stock mínimo.");
    }

    JOptionPane.showMessageDialog(null, "Stock actualizado");
    return;
}
}

    JOptionPane.showMessageDialog(null, "Material no encontrado");
}
}

```

Clase ListarMateriales

```

package com.mycompany.sistemagestiondealmacen;
import javax.swing.JOptionPane;

public class ListarMateriales {
    public void listar(Material[] inventario, int contador) {

        if (contador == 0) {

```

```
        JOptionPane.showMessageDialog(null, "No hay materiales  
registrados");
```

```
        return;
```

```
    }
```

```
String[] opciones = {
```

```
    "Ver todo",
```

```
    "Materiales de Obra",
```

```
    "Materiales Consumibles",
```

```
    "Herramientas"
```

```
};
```

```
String filtro = (String) JOptionPane.showInputDialog(
```

```
    null,
```

```
    "Seleccione tipo de listado:",
```

```
    "Filtro",
```

```
    JOptionPane.QUESTION_MESSAGE,
```

```
    null,
```

```
    opciones,
```

```
    opciones[0]
```

```
);
```

```
String lista = "=== INVENTARIO ===\n\n";
```

```
for (int i = 0; i < contador; i++) {
```

```
    if (!filtro.equals("Ver todo") &&
```

```
        !inventario[i].getCategoria().equalsIgnoreCase(filtro)) {
```

```
        continue;
```

```
    }
```



```

        Nombre: "" + inventario[i].getNombre() +
            "\nCantidad: " + inventario[i].getCantidad() +
            "\nUnidad de medida: " + inventario[i].getUnidadMedida() +
            "\nCategoría: " + inventario[i].getCategoria() +
            "\nStock mínimo: " + inventario[i].getStockMinimo() +
            "\nDesperdicio: " + inventario[i].getDesperdicio() +
            "\nÚltima modificación: " +
inventario[i].getUltimaActualizacion()
    );

    return;
}
}

JOptionPane.showMessageDialog(null, "Material no encontrado");
}
}

```

Clase Responsable

```
package com.mycompany.sistemagestiondealmacen;
```

```

public class Responsable {

    private final String nombre;
    private final String apellido;
    private final String dni;

    public Responsable(String nombre, String apellido, String dni) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.dni = dni;
    }
}

```

```

    }

    public String getNombreCompleto() {
        return nombre + " " + apellido;
    }

    public String getDni() {
        return dni;
    }
}

```

Clase Historial

```
package com.mycompany.sistemagestiondealmacen;
```

```
import javax.swing.JOptionPane;
```

```

public class Historial {

    private final String[] acciones = new String[200];
    private int contador = 0;

    public void registrarAccion(String texto) {
        if (contador < acciones.length) {
            acciones[contador] = texto;
            contador++;
        }
    }

    public void mostrarHistorial() {
        if (contador == 0) {

```

```

        JOptionPane.showMessageDialog(null, "No hay registros en el
historial");
        return;
    }

    StringBuilder sb = new StringBuilder("=== HISTORIAL DE ACCIONES
===\n\n");

    for (int i = 0; i < contador; i++) {
        sb.append(acciones[i]).append("\n");
    }

    JOptionPane.showMessageDialog(null, sb.toString());
}
}

```

12. Conclusiones

Crear el Sistema de Control de Materiales demostró cuán vital es poner los procesos internos de una constructora en formato digital. Tras estudiarlo bien, vimos que manejar todo a mano con hojas de cálculo y papeles causaba fallos, datos repetidos, información extraviada, no saber qué pasó con algo y demoras al decidir.

Poner en marcha este programa hecho en Java da una forma práctica y sencilla de juntar todos los registros de materiales, vigilar lo que hay, anotar cuándo entra algo, cuánto se usa y cuánto se tira, e incluso avisar si quedan pocas existencias. Usar ventanas simples de JOptionPane ayudó a hacer una herramienta fácil de entender y que el personal de bodega y obra pudiera usar sin problemas.

Además, el programa sirvió para hacer más claras las operaciones, seguir mejor el rastro de lo que se mueve, y saber quién hizo qué gracias al registro de quién entra y el historial de actividades. Estos logros indican que tener un sistema digital es fundamental para ser más productivos, gastar menos plata y usar mejor los medios en las obras de construcción.

Para terminar, este proyecto logró sin problemas lo que se propuso: hacer digital la gestión de inventario, bajar los errores, vigilar el

desperdicio, ayudar a planear mejor y respaldar las decisiones importantes que toma la empresa constructora.

13. Recomendaciones

- Ampliar el sistema con una base de datos real

El sistema funciona con arreglos internos, pero para un uso empresarial continuo se recomienda migrar a una base de datos como MySQL o PostgreSQL para almacenar grandes volúmenes de información de manera persistente y segura.

- Optimizar la interfaz gráfica

Migrar a JavaFX o desarrollar una interfaz web para una experiencia visual más moderna y con navegación más fluida.

- Ampliar el historial de acciones

Incluir filtros por usuario, fecha o acción facilitaría auditorías internas y análisis posteriores.

14. Referencias Bibliográficas

Java - methods. (s. f.). https://www.tutorialspoint.com/java/java_methods.htm

Chapter 8. Classes. (2025, 16 septiembre).

<https://docs.oracle.com/javase/specs/jls/se17/html/jls-8.html>

GeeksforGeeks. (2025, 24 septiembre). *Java OOP(Object Oriented*

Programming) concepts. GeeksforGeeks.

<https://www.geeksforgeeks.org/java/object-oriented-programming-oops-concept-in-java/>

JOptionPane (Java Platform SE 8). (2025, 20 octubre).

<https://docs.oracle.com/javase/8/docs/api/javax/swing/JOptionPane.html#:~:text=,section%20in%20The%20Java%20Tutoria>

Tutorial de diagrama de clases UML. (2025, 16 septiembre). Lucidchart.

<https://www.lucidchart.com/pages/es/tutorial-de-diagrama-de-clases-uml#:~:text=Los%20diagramas%20de%20clases%20son,tu%20propio%20diagrama%20de%20clases>

15. Anexos

<https://youtu.be/qvVTUc4MwCc>